Transforming Raw Data into gold.dim_customers

A complete, line-by-line walkthrough of a medallion-architecture SQL transformation pipeline — from messy source systems to a clean, business-ready customer dimension.

What this guide covers

This document reconstructs the full transformation logic behind the **gold_dim_customers** table you already have, showing every SQL step needed to produce it from three typical source systems: a CRM customer table, an ERP demographic table, and an ERP location table. Each section pairs runnable T-SQL with a plain-language explanation of why the step exists.

Target table

gold.dim_customers — customer_key, customer_id, customer_number, first_name, last_name, country, marital_status, gender, birthdate, create_date

1 Architecture Overview

The **medallion architecture** organizes data into three progressively refined layers. Each layer has one job, which keeps the transformation logic easy to test and debug.

Layer	Job	What happens here
Bronze	Land the data	Raw extracts loaded as-is from CRM and ERP source files, no changes
Silver	Clean the data	Trim text, fix casing, remove duplicates, standardize codes, validate dates
Gold	Model the data	Join cleaned tables, resolve conflicts, generate surrogate keys, rename for business users

Three sources feed one dimension

gold.dim_customers is not built from a single table. It merges three source systems, each contributing different columns, and Customer Relationship Management (CRM) data is treated as the trusted “master” source whenever two systems disagree.

Source table	System	Contributes
crm_cust_info	CRM	customer id, customer key/number, first/last name, marital status, gender, create date
erp_cust_az12	ERP	birthdate, a secondary gender field used as a fallback
erp_loc_a101	ERP	country

Why three layers instead of one big query?

Why not just query the sources directly? Source systems store gender as free text (‘M’, ‘MALE’, ‘ ’, NULL...), keep duplicate customer records, and use cryptic country codes. The silver layer exists specifically to fix these before anyone builds a report on top of them.

2 Source Systems (Bronze Layer)

The bronze tables are loaded unmodified from the source files. Below is the shape of each one before any cleaning — this is the messy starting point the silver layer has to fix.

bronze.crm_cust_info

Column	Sample raw value	Problem
cst_id	11000	Occasional duplicates from re-imports
cst_key	AW00011000	Clean, used as customer_number
cst_firstname	Jon	Leading/trailing spaces
cst_lastname	yang	Inconsistent casing
cst_marital_status	S / M / NULL	Abbreviated codes, some blank
cst_gndr	M / F / '' / NULL	Inconsistent codes and blanks
cst_create_date	2025-10-06	Used to pick the newest duplicate record

bronze.erp_cust_az12

Column	Sample raw value	Problem
cid	NAS11000 / 11000	Sometimes prefixed with 'NAS', sometimes not
bdate	1971-10-06 / 2071-01-01	Some birthdates are impossibly in the future
gen	Male / M / F / NULL	Different spelling/casing than CRM

bronze.erp_loc_a101

Column	Sample raw value	Problem
cid	11000 (with dashes stripped elsewhere)	Needs matching format to crm_cust_info.cst_key
cntry	US / USA / DE / '' / NULL	Country codes, not full names

> **Bronze rule of thumb**
>
> Nothing in this layer is trusted yet. Column names still use terse source-system abbreviations (cst_, cid, gen) because renaming and cleaning are silver-layer jobs.

3 Silver: crm_cust_info

This step removes duplicate customers, trims whitespace, and converts abbreviated codes into readable labels. The transformation is wrapped in a view so it always reflects the latest bronze data.

Step 3.1 — Keep only the latest record per customer

```
WITH ranked_customers AS (
    SELECT
        cst_id,
        cst_key,
        cst_firstname,
        cst_lastname,
        cst_marital_status,
        cst_gndr,
        cst_create_date,
        ROW_NUMBER() OVER (
            PARTITION BY cst_id
            ORDER BY cst_create_date DESC
        ) AS row_flag
    FROM bronze.crm_cust_info
    WHERE cst_id IS NOT NULL
)
```

PARTITION BY cst_id groups every row belonging to the same customer without collapsing them, and **ORDER BY cst_create_date DESC** ranks the newest record first within each group. Row 1 in every partition is the one we keep — this is a window function doing the job GROUP BY physically can't, since we still need every column, not just an aggregate.

Step 3.2 — Clean text and standardize codes

```
SELECT
    cst_id AS customer_id,
    cst_key AS customer_number,
    TRIM(cst_firstname) AS first_name,
    TRIM(cst_lastname) AS last_name,
    CASE UPPER(TRIM(cst_marital_status))
        WHEN 'S' THEN 'Single'
        WHEN 'M' THEN 'Married'
        ELSE 'n/a'
    END AS marital_status,
    CASE UPPER(TRIM(cst_gndr))
        WHEN 'M' THEN 'Male'
        WHEN 'F' THEN 'Female'
        ELSE 'n/a'
    END AS gender,
    cst_create_date AS create_date
FROM ranked_customers
WHERE row_flag = 1;
```

Each transformation targets one problem: **TRIM()** removes stray whitespace so ‘Jon ’ and ‘Jon’ aren't treated as different values; **UPPER()** makes the CASE comparison case-insensitive; and the trailing **ELSE 'n/a'** guarantees a

NULL or unexpected code never silently disappears from a report.

Result: silver.crm_cust_info

customer_id	customer_number	first_name	marital_status	gender
11000	AW00011000	Jon	Married	Male
11001	AW00011001	Eugene	Single	Male
11002	AW00011002	Ruben	Married	Male

Checkpoint

Notice marital_status and gender are now full English words, matching what appears in gold.dim_customers — that mapping happens once, here, instead of being repeated in every downstream report.

4 Silver: erp_cust_az12

This source contributes birthdate and a secondary gender field. Two problems need fixing: a stray 'NAS' prefix on the customer id, and birthdates that were entered in the future.

Step 4.1 — Normalize the id and validate birthdate

```
SELECT
    CASE
        WHEN cid LIKE 'NAS%' THEN SUBSTRING(cid, 4, LEN(cid))
        ELSE cid
    END AS customer_id_clean,
    CASE
        WHEN bdate > GETDATE() THEN NULL
        ELSE bdate
    END AS birthdate,
    CASE UPPER(TRIM(gen))
        WHEN 'MALE' THEN 'Male'
        WHEN 'M' THEN 'Male'
        WHEN 'FEMALE' THEN 'Female'
        WHEN 'F' THEN 'Female'
        ELSE 'n/a'
    END AS gender_erp
FROM bronze.erp_cust_az12;
```

SUBSTRING(cid, 4, LEN(cid)) strips the first three characters ('NAS') whenever they appear, so this table's id lines up with cst_key/customer_number from CRM. The birthdate check simply throws out anything later than today — a future birthdate can never be correct, so NULL is safer than a wrong date.

Result: silver.erp_cust_az12

customer_id_clean	birthdate	gender_erp
11000	1971-10-06	Male
11001	1976-05-10	Male
11002	1971-02-09	Male

5 Silver: erp_loc_a101

The location table maps each customer to a country, but stores it as an inconsistent abbreviation. This step turns codes into the full country names seen in gold.dim_customers.

Step 5.1 — Standardize country codes

```
SELECT
    REPLACE(cid, '-', '') AS customer_id_clean,
    CASE TRIM(cntry)
        WHEN 'DE' THEN 'Germany'
        WHEN 'US' THEN 'United States'
        WHEN 'USA' THEN 'United States'
        WHEN '' THEN 'n/a'
        WHEN NULL THEN 'n/a'
        ELSE TRIM(cntry)
    END AS country
FROM bronze.erp_loc_a101;
```

REPLACE(cid, '-', '') removes dashes so the id format matches the other two tables — this is the join key that will stitch all three sources together in the gold layer. The CASE expression covers known code variants first, then falls back to whatever value was already there for countries that don't need remapping.

> **Why the id cleanup matters so much**
>
> A join only works when the join key looks identical on both sides. This is why every silver step that touches an id column also standardizes its format — removing prefixes, dashes, or leading zeros — before the gold layer ever runs a JOIN.

6 Gold: Building dim_customers

With all three silver tables clean and using matching id formats, the gold layer joins them, resolves the one remaining conflict (two different gender fields), generates a surrogate key, and renames columns for business users.

Step 6.1 — Join the three silver tables

```
SELECT
    ci.customer_id,
    ci.customer_number,
    ci.first_name,
    ci.last_name,
    ci.marital_status,
    ci.gender AS gender_crm,
    ca.gender_erp,
    ca.birthdate,
    la.country,
    ci.create_date
FROM silver.crm_cust_info ci
LEFT JOIN silver.erp_cust_az12 ca
    ON ci.customer_number = ca.customer_id_clean
LEFT JOIN silver.erp_loc_a101 la
    ON ci.customer_number = la.customer_id_clean;
```

LEFT JOIN is used twice so that every CRM customer is kept even if a matching ERP record is missing — losing customers because of an incomplete demographic feed would be worse than showing a blank birthdate or country for a few rows.

Step 6.2 — Resolve the gender conflict

```
SELECT
    ...
    CASE
        WHEN gender_crm <> 'n/a' THEN gender_crm
        ELSE COALESCE(gender_erp, 'n/a')
    END AS gender,
    ...
FROM joined_customers;
```

CRM is treated as the trusted source because it's entered directly by sales reps at the point of contact. The rule reads: *use the CRM gender whenever it's actually filled in; only fall back to the ERP value when CRM has nothing.*
COALESCE then guarantees the column is never blank — it returns the first non-NULL value from its list.

Step 6.3 — Generate the surrogate key

gold.dim_customers exposes a customer_key column that has nothing to do with any source system — it's a clean, sequential integer generated purely for warehouse use, so fact tables can join to it without ever depending on a source id that might change format again in the future.

```
SELECT
    ROW_NUMBER() OVER (ORDER BY customer_id) AS customer_key,
    customer_id,
    customer_number,
    first_name,
    last_name,
    country,
    marital_status,
    gender,
    birthdate,
    create_date
FROM resolved_customers;
```

With no PARTITION BY, **ROW_NUMBER() OVER (ORDER BY customer_id)** treats the whole table as a single window and numbers every row 1, 2, 3... in customer_id order. This is the same window-function pattern used earlier for deduplication — only this time there's nothing to partition, because every row is already unique and every row gets a key.

> **Why not just use customer_id as the key?**
>
> A surrogate key insulates the warehouse from source-system changes: if the CRM ever changes its id format, every fact table that joins on customer_key keeps working untouched, because customer_key was never derived from that source id in the first place.

7 Worked Example: One Customer, Start to Finish

Abstract rules are easier to trust once you watch a single, real-looking row travel through every layer. Below is customer **11000 / Jon Yang** at each stage of the pipeline.

Bronze — as-is from each source

Source	Raw row
crm_cust_info	cst_id=11000, cst_key=AW00011000, cst_firstname=' Jon', cst_gndr='M', cst_marital_status='M'
erp_cust_az12	cid='NAS11000', bdate=1971-10-06, gen='Male'
erp_loc_a101	cid='11-000', cntry='US'

Silver — cleaned independently

Source	Cleaned row
silver.crm_cust_info	customer_id=11000, first_name='Jon', gender='Male', marital_status='Married'
silver.erp_cust_az12	customer_id_clean=11000, birthdate=1971-10-06, gender_erp='Male'
silver.erp_loc_a101	customer_id_clean=11000, country='United States'

Notice all three customer_id_clean / customer_number values are now the digits **11000** with no prefix and no dashes — this is what makes the gold-layer joins line up correctly.

Gold — joined, resolved, keyed

Column	Final value	Where it came from
customer_key	1	ROW_NUMBER() OVER (ORDER BY customer_id)
customer_id	11000	crm.customer_id
customer_number	AW00011000	crm.customer_number
first_name	Jon	crm.first_name (trimmed)
country	United States	erp_l.country
marital_status	Married	crm.marital_status
gender	Male	crm.gender (CRM wins, already non-blank)
birthdate	1971-10-06	erp_a.birthdate

Tracing the gender rule

gender resolves to 'Male' here because the CRM value was already valid — the ERP fallback (gender_erp) never even gets used for this row. The fallback only kicks in for customers where CRM's gender was blank or 'n/a'.

A row where the fallback matters

Imagine a different customer, 11050, whose CRM record has cst_gndr = NULL but whose ERP record has gen = 'F'. The CASE/COALESCE logic evaluates like this:

```
-- crm.gender = 'n/a' (NULL mapped by the CASE in silver.crm_cust_info)
-- gender_erp = 'Female'
CASE
    WHEN crm.gender <> 'n/a' THEN crm.gender -- false, skip
    ELSE COALESCE(erp_a.gender_erp, 'n/a') -- returns 'Female'
END
-- result: gender = 'Female'
```

This is exactly why the fallback exists: without it, any customer missing a CRM gender would show 'n/a' in the gold table even though the ERP system had the answer all along.

8 Incremental Loads & Idempotency

gold.dim_customers is defined as a **VIEW**, so it always reflects the current bronze and silver data with no separate refresh step. Some warehouses instead materialize it as a table for performance — if you do that, the load needs to be safe to re-run.

Pattern: full reload with TRUNCATE + INSERT

```
TRUNCATE TABLE gold.dim_customers_tbl;

INSERT INTO gold.dim_customers_tbl
SELECT * FROM gold.dim_customers; -- the view built earlier
```

Simple and predictable: every run produces an identical result from the same source data. This is **idempotent** — running it twice in a row causes no harm, unlike an INSERT with no TRUNCATE, which would duplicate every row on a second run.

Pattern: MERGE for large tables

```
MERGE gold.dim_customers_tbl AS target
USING gold.dim_customers AS source
ON target.customer_id = source.customer_id
WHEN MATCHED THEN
    UPDATE SET
        target.country = source.country,
        target.marital_status = source.marital_status,
        target.gender = source.gender,
        target.birthdate = source.birthdate
WHEN NOT MATCHED THEN
    INSERT (customer_key, customer_id, customer_number, first_name,
            last_name, country, marital_status, gender, birthdate, create_date)
    VALUES (source.customer_key, source.customer_id, source.customer_number,
            source.first_name, source.last_name, source.country,
            source.marital_status, source.gender, source.birthdate, source.create_date);
```

> **Which pattern should you use?**
>
> TRUNCATE + INSERT is simplest and fine for a dimension this size (tens of thousands of rows). MERGE is worth the added complexity once the table is too large to fully rebuild on every run, since it only touches rows that actually changed.

9 Full Assembled Script

Putting every step together as a single view. This is the production version of the logic built section by section above.

```
CREATE OR ALTER VIEW gold.dim_customers AS
WITH ranked_crm AS (
    SELECT *,
        ROW_NUMBER() OVER (
            PARTITION BY cst_id ORDER BY cst_create_date DESC
        ) AS row_flag
    FROM bronze.crm_cust_info
    WHERE cst_id IS NOT NULL
),
crm AS (
    SELECT
        cst_id AS customer_id,
        cst_key AS customer_number,
        TRIM(cst_firstname) AS first_name,
        TRIM(cst_lastname) AS last_name,
        CASE UPPER(TRIM(cst_marital_status))
            WHEN 'S' THEN 'Single' WHEN 'M' THEN 'Married'
            ELSE 'n/a' END AS marital_status,
        CASE UPPER(TRIM(cst_gndr))
            WHEN 'M' THEN 'Male' WHEN 'F' THEN 'Female'
            ELSE 'n/a' END AS gender,
        cst_create_date AS create_date
    FROM ranked_crm WHERE row_flag = 1
),
```

```
erp_a AS (
    SELECT
        CASE WHEN cid LIKE 'NAS%' THEN SUBSTRING(cid,4,LEN(cid))
             ELSE cid END AS customer_id_clean,
        CASE WHEN bdate > GETDATE() THEN NULL ELSE bdate END AS birthdate,
        CASE UPPER(TRIM(gen))
            WHEN 'MALE' THEN 'Male' WHEN 'M' THEN 'Male'
            WHEN 'FEMALE' THEN 'Female' WHEN 'F' THEN 'Female'
            ELSE 'n/a' END AS gender_erp
    FROM bronze.erp_cust_az12
),
erp_l AS (
    SELECT
        REPLACE(cid, '-', '') AS customer_id_clean,
        CASE TRIM(cntry)
            WHEN 'DE' THEN 'Germany'
            WHEN 'US' THEN 'United States' WHEN 'USA' THEN 'United States'
            WHEN '' THEN 'n/a'
            ELSE TRIM(cntry) END AS country
    FROM bronze.erp_loc_a101
)
```

Final SELECT of the view

```
SELECT
    ROW_NUMBER() OVER (ORDER BY crm.customer_id) AS customer_key,
    crm.customer_id,
    crm.customer_number,
    crm.first_name,
    crm.last_name,
    erp_l.country,
    crm.marital_status,
    CASE
        WHEN crm.gender <> 'n/a' THEN crm.gender
        ELSE COALESCE(erp_a.gender_erp, 'n/a')
    END AS gender,
    erp_a.birthdate,
    crm.create_date
FROM crm
LEFT JOIN erp_a ON crm.customer_number = erp_a.customer_id_clean
LEFT JOIN erp_l ON crm.customer_number = erp_l.customer_id_clean;
```

How the pieces fit together

This is one CREATE VIEW statement built from four CTEs chained together: two feed the final join (crm, and the two erp CTEs), and one (ranked_crm) exists purely to support the deduplication step. Reading top to bottom mirrors the section order of this guide.

10 Data Quality Validation

Before trusting a gold table, run checks that would catch the exact problems the silver layer was designed to fix. Each check below should return zero rows.

Check 1 — No duplicate customer keys

```
SELECT customer_key, COUNT(*) AS occurrences
FROM gold.dim_customers
GROUP BY customer_key
HAVING COUNT(*) > 1;
```

Check 2 — No unexpected gender values

```
SELECT DISTINCT gender
FROM gold.dim_customers
WHERE gender NOT IN ('Male', 'Female', 'n/a');
```

Check 3 — No future birthdates

```
SELECT customer_id, birthdate
FROM gold.dim_customers
WHERE birthdate > GETDATE();
```

Check 4 — Row counts match expectations

```
SELECT
    (SELECT COUNT(DISTINCT cst_id) FROM bronze.crm_cust_info) AS distinct_source_customers,
    (SELECT COUNT(*) FROM gold.dim_customers) AS gold_row_count;
```

Reading the results

Check 1 confirms the ROW_NUMBER() surrogate key generation didn't accidentally produce ties. Check 2 confirms the CASE/COALESCE gender logic covers every real value and nothing slipped through as raw source text. Check 3 re-verifies the birthdate guard survived the join. Check 4 confirms the LEFT JOINs didn't fan out rows — the gold row count should exactly equal the distinct customer count from CRM, since each join key is unique per customer in every silver table.

> **Troubleshooting tip**
>
> If Check 4 shows more gold rows than source customers, the most common cause is a duplicate join key on the ERP side — meaning erp_cust_az12 or erp_loc_a101 still has more than one row per customer_id_clean, and the LEFT JOIN is fanning out.

Check	Target	Catches
Duplicate keys	0 rows	Broken surrogate-key generation
Unexpected gender	0 rows	A CASE branch that missed a code
Future birthdates	0 rows	A bad date that slipped past the guard
Row count match	Equal	Join fan-out from duplicate ERP rows

11 Technique Cheat Sheet

Technique	Used for	Where in this guide
ROW_NUMBER() OVER (PARTITION BY ...)	Keep only the newest duplicate row	Silver: crm_cust_info dedup
ROW_NUMBER() OVER (ORDER BY ...)	Generate a sequential surrogate key	Gold: customer_key
TRIM() / UPPER()	Normalize text before comparing	Silver: all three tables
CASE WHEN ... THEN ... ELSE	Map codes to readable labels; validate values	Silver + gold gender logic
COALESCE()	Fall back to a second source when the first is blank	Gold: gender resolution
LEFT JOIN	Keep every CRM customer even without ERP match	Gold: final assembly
CTE (WITH ... AS (...))	Break a long transformation into named, readable steps	Every silver & gold step

Glossary

Term	Meaning
Surrogate key	A generated key with no business meaning, used only inside the warehouse
Natural key	An identifier from the source system (customer_id) that could change or repeat
CTE	Common Table Expression: a named, temporary result set defined with WITH
Window function	A calculation over a set of rows that keeps every row, unlike GROUP BY
Medallion architecture	Bronze/Silver/Gold layering pattern for progressively cleaner data
Fan-out	Unwanted row duplication caused by a join matching more than one row per key

Column dictionary for gold.dim_customers

Column	Source	Notes
customer_key	Generated	Surrogate key, sequential integer
customer_id	crm_cust_info.cst_id	Natural key from CRM
customer_number	crm_cust_info.cst_key	Business-facing customer code (e.g. AW00011000)
first_name / last_name	crm_cust_info	Trimmed
country	erp_loc_a101.cntry	Standardized to full country name
marital_status	crm_cust_info.cst_marital_status	Standardized to Single / Married / n/a
gender	crm_cust_info.cst_gndr, fallback erp_cust_az12.gen	CRM wins ties; ERP fills gaps
birthdate	erp_cust_az12.bdate	Future dates nulled out
create_date	crm_cust_info.cst_create_date	Original CRM record creation date

Closing note

Every column in gold.dim_customers can be traced back to exactly one bronze source and one transformation rule in this guide. That traceability is the whole point of building the pipeline in layers instead of one large, opaque query.

12 Common Pitfalls

These are the mistakes that most often break a transformation like this one in practice, along with the fix already baked into the pipeline above.

Pitfall	Symptom	Fix used in this guide
Comparing text without TRIM/UPPER	'M ' and 'M' treated as different codes	TRIM(UPPER(...)) before every CASE comparison
Deduplicating with DISTINCT instead of ROW_NUMBER	Picks an arbitrary row, not the newest	ROW_NUMBER() OVER (PARTITION BY ... ORDER BY create_date DE
Mismatched join key formats	Rows silently drop out of a LEFT JOIN as NULLs	Strip prefixes/dashes from ids in every silver step
Using INNER JOIN for optional data	Customers missing an ERP row disappear entirely	LEFT JOIN keeps every CRM customer
No ELSE in a CASE expression	Unexpected codes become silent NULLs	Every CASE here ends in ELSE 'n/a'
Deriving a key from a source id	Key changes if the source system changes format	customer_key is a generated ROW_NUMBER(), independent of source id

One-sentence summary

Most of the SQL in this guide is short. The value is in the ordering: clean before you join, validate before you trust, and never let a source system's quirks leak past the silver layer.